

Algorithms 1

Basics (Meta-stuff)

Generally, it seems more like algs1 was about proving the code (like start and end pointers and such) while algs2 is proving the algorithm.

Time complexity

Some formal definitions exists, you could know them, but generally it's enough to say that:

$$\begin{aligned}g(n) \in o(f(n)) &\Leftrightarrow g < f \\g(n) \in O(f(n)) &\Leftrightarrow g \leq f \\g(n) \in \Theta(f(n)) &\Leftrightarrow g = f \\g(n) \in \Omega(f(n)) &\Leftrightarrow g \geq f \\g(n) \in \omega(f(n)) &\Leftrightarrow g > f\end{aligned}$$

(well, as $n \rightarrow \infty$, and ignoring constant factors)

Can end up with recursive time-taken like $T(n) = k_1 + k_2n + 2T(n/2)$. Then can do: 1. Substitute into itself and spot a pattern 2. *Diagram Time* 3. Master Theorem (see below) 4. Guess (include constants like k_1 , etc.) and then substitute it

Master Theorem

If $T(n) = aT(\frac{n}{b}) + f(n)$ (and $T(1) = 1$, duh):

Let $K = \log_b(a)$:

Case	i.e.		Dominated by
$f(n) \in O(n^{K-\varepsilon})$	$f \lesssim n^{(<K)}$	$T(n) \in \Theta(n^K)$	The recursion bit
$f(n) \in \Theta(n^K)$	$f \sim n^K$	$T(n) \in \Theta(n^K \log n)$	(Equal)
$f(n) \in \Omega(n^{K+\varepsilon})^*$	$f \gtrsim n^{(>K)^*}$	$T(n) \in O(f(n))^*$	The work on the root node

*if the regularity condition holds (must check?): $f(\frac{n}{b}) \leq cf(n)$ for some $c < 1$ (for all sufficiently large n , same c for each). (Needed so it's geometric sequence proper and not oscillating thing as then child may be more than parent)

Proofs of correctness

Super annoying. Induction most of the time: 1. Initialisation: statement true before a loop 2. Maintenance: statement invariant over the loop 3. Termination: statement must be true and end of loop; hope that it help you prove correctness

Dynamic Programming

Useful for optimisation problems (e.g. VM Hosting Problem). Effectively recursion plus cache. Requires: - Optimal substructure (best solution uses a best solution of one of the subproblems) - Overlapping subproblems (so can reuse results)

Memoise previous results. Top-down (recursive, cache) or bottom-up (iterative, build up table from 0). Top-

down easier to code but requires stack space (reuse order?). Bottom-up doesn't require stack space but still requires the table (even for potentially-useless bits of the problem).

Greedy Algorithm

Faster than DP. Make locally optimal choices (and these don't rely on further computation) and prove that combining these yields a globally optimal solution. Effectively, greedy algorithms are DP where we choose which subproblem will be the optimal one (though might not do reuse).